

Passeio Turístico Máximo — Caminho Mais Longo

Análise e Projeto de Algoritmos

Alunos: Yves Luan do Rosário Moura e Patrick César Santos Silva

Disciplina: Análise e Projeto de Algoritmos · **Prof.:** George Lima · **Data:** 01/07/2026

Parte A — Complexidade e Redução Polinomial

Para tratar a dificuldade do problema no sentido de NP, considero a versão de decisão do Caminho Mais Longo e faço uma redução a partir do Caminho Hamiltoniano, que é NP-Completo.

Caminho Mais Longo (decisão). Dado $G=(V,E)$ não direcionado com pesos $w(u,v)>0$, vértices S e D e um inteiro k : existe caminho simples de S a D com soma de pesos $\geq k$?

Caminho Hamiltoniano (decisão). Dado $G=(V,E)$ não direcionado, existe caminho simples que passa por todos os $n=|V|$ vértices uma única vez?

Construção da redução

Seja $G=(V,E)$ uma instância qualquer do Caminho Hamiltoniano, com $n=|V|$. Construo uma instância (G',S,D,k) do Caminho Mais Longo da seguinte forma:

- G' tem os mesmos vértices e as mesmas arestas de G ;
- toda aresta de G' recebe peso 1;
- $k = n - 1$;
- como o Hamiltoniano não fixa início e fim, considero cada par ordenado (S,D) de vértices distintos. São $n(n-1)$ pares; o Hamiltoniano tem resposta "sim" se, e somente se, algum desses pares tiver resposta "sim" no Caminho Mais Longo.

A construção troca o peso de cada aresta e gera $O(n^2)$ pares de extremos, logo é feita em tempo polinomial.

Equivalência das respostas

($\Rightarrow$) Suponha que G tem caminho hamiltoniano $v_1, \dots, v_n$. Esse caminho usa $n-1$ arestas e, como cada aresta vale 1, soma $n-1 = k$. Tomando $S=v_1$ e $D=v_n$, ele é um caminho simples em G' com soma $\geq k$. Logo o Caminho Mais Longo responde "sim".

($\Leftarrow$) Suponha que existe caminho simples de soma $\geq n-1$ em G' para algum par (S,D) . Como toda aresta vale 1, esse caminho tem pelo menos $n-1$ arestas. Mas um caminho simples num grafo de n vértices tem no máximo $n-1$ arestas, pois não repete vértice. Então ele tem exatamente $n-1$ arestas e, portanto, n vértices distintos — ou seja, passa por todos. Esse é um caminho hamiltoniano em G . Logo o Hamiltoniano responde "sim".

A transformação é polinomial e preserva a resposta nos dois sentidos. Portanto Caminho Hamiltoniano $\leq_p$ Caminho Mais Longo, e como o Hamiltoniano é NP-Completo, o Caminho Mais Longo é **NP-Difícil**. Além disso, dado um caminho candidato, verifico em tempo linear se ele é simples e se atinge k, então o problema está em NP; juntando as duas coisas, ele é **NP-Completo**.

Observação. O Caminho Mais Curto é resolvido eficientemente (Dijkstra) porque qualquer subcaminho de um caminho mínimo também é mínimo, o que permite construir a solução localmente. No Caminho Mais Longo simples isso não vale: a restrição de não repetir vértice cria dependência entre as escolhas, e decidir localmente não garante o ótimo. Essa é a raiz da dificuldade.

Parte B — Implementação e Experimentos

Implementei as duas abordagens em Python. Os grafos usados são completos, com pesos inteiros sorteados entre 1 e 100. Deixei a semente do sorteio fixa (`random.seed(42)`) pra poder repetir o experimento e dar sempre o mesmo resultado. Em todos os testes a origem é o vértice 0 e o destino é o vértice $n-1$.

Solução exata (backtracking)

A ideia é um DFS recursivo que testa todos os caminhos possíveis de S até D e guarda o de maior peso. O detalhe importante é desmarcar o vértice na volta da recursão, para que ele possa ser usado de novo em outros caminhos. Coloquei também um limite de tempo de 20 segundos, já que para grafos grandes a busca não termina.

```
def resolver_exato(g, n, origem, destino, limite=20.0):
    # backtracking: testa todos os caminhos simples de origem ate destino
    global melhor_peso, melhor_caminho, inicio_tempo
    melhor_peso = -1
    melhor_caminho = None
    visitado = [False] * n
    inicio_tempo = time.perf_counter()

    def buscar(atual, peso, caminho):
        global melhor_peso, melhor_caminho
        # se demorar demais, desiste (grafos grandes sao inviaveis)
        if time.perf_counter() - inicio_tempo > limite:
            raise TimeoutError
        if atual == destino:
            if peso > melhor_peso:
                melhor_peso = peso
                melhor_caminho = list(caminho)
            return
        for prox in range(n):
            if prox != atual and not visitado[prox] and g[atual][prox] > 0:
                visitado[prox] = True
                caminho.append(prox)
                buscar(prox, peso + g[atual][prox], caminho)
                caminho.pop()
                visitado[prox] = False # desmarca pra liberar pra outras rotas

    visitado[origem] = True
```

```
buscar(origem, 0, [origem])
return melhor_peso, melhor_caminho
```

Solução gulosa (heurística)

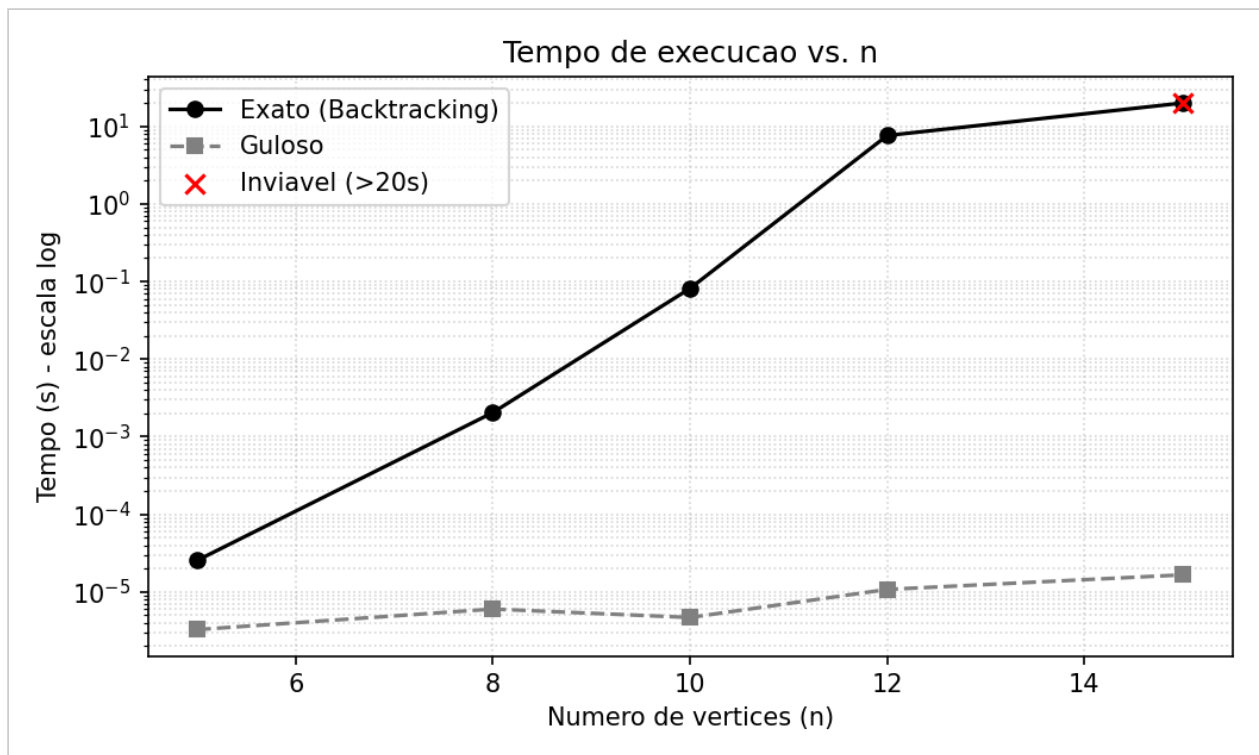
O guloso é bem mais simples: começa na origem e vai sempre pro vizinho ainda não visitado que tiver a maior aresta. Para quando chega no destino, ou quando fica preso sem ter para onde ir (beco sem saída).

```
def resolver_guloso(g, n, origem, destino):
    # sempre vai pro vizinho de maior peso que ainda nao foi visitado
    visitado = [False] * n
    visitado[origem] = True
    atual = origem
    peso = 0
    caminho = [origem]
    while atual != destino:
        escolhido = -1
        maior = -1
        for prox in range(n):
            if prox != atual and not visitado[prox] and g[atual][prox] > maior:
                maior = g[atual][prox]
                escolhido = prox
        if escolhido == -1:
            return None, caminho # ficou preso num beco sem saida
        visitado[escolhido] = True
        peso += maior
        caminho.append(escolhido)
        atual = escolhido
    return peso, caminho
```

O restante do código (geração dos grafos aleatórios, laço dos experimentos e produção dos gráficos) está no arquivo `longest_path.py` entregue junto com este relatório.

Tempo de execução

n	Exato	Guloso
5	23 us	3 us
8	1.82 ms	7 us
10	107.74 ms	7 us
12	9.76 s	12 us
15	— (parou em 20 s)	19 us

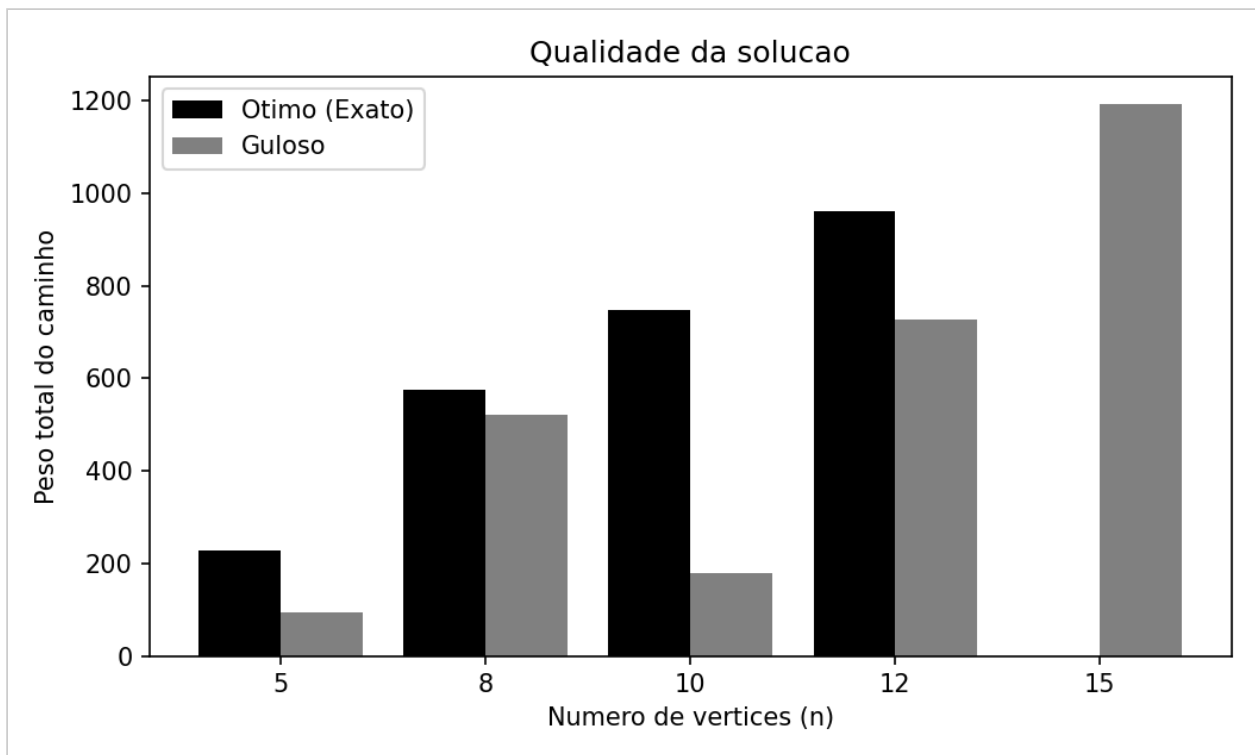


O eixo do tempo está em escala logarítmica. Observa-se que o tempo do exato sobe quase numa reta nesse gráfico, o que indica crescimento muito rápido (num grafo completo o número de caminhos de S a D é da ordem de $(n-2)!$). Já o guloso fica praticamente constante, perto de zero.

Onde a força bruta deixa de ser viável: com $n=12$ o exato já levou quase 10 segundos na minha máquina. Com $n=15$ tive que interromper a execução, porque passou de 20 segundos sem terminar. Então é mais ou menos nesse ponto (entre 12 e 15) que a solução exata vira algo impraticável. O guloso resolve o mesmo $n=15$ em alguns microssegundos.

Qualidade da solução

n	Ótimo	Guloso	Quanto pior
5	227	95	58.1%
8	574	521	9.2%
10	747	180	75.9%
12	960	726	24.4%
15	não terminou	1192	—



Olhando os casos em que a força bruta conseguiu terminar, o guloso ficou de 9% a 76% pior que o ótimo, sem nenhum padrão. Os casos $n=5$ e $n=10$ são os mais chamativos: o guloso pega logo de cara a aresta mais pesada saindo da origem, que acaba levando direto ao destino, e o caminho termina com pouquíssimos vértices, deixando de fora boa parte do grafo.

O caso $n=10$ deixa isso bem claro. O caminho ótimo encontrado pelo backtracking foi $0 \rightarrow 1 \rightarrow 7 \rightarrow 8 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 9$, passando por todos os 10 vértices e somando peso 747. Já o guloso saiu de 0, foi direto para o vértice 7 (que tinha a aresta mais pesada) e de lá caiu no destino 9, parando no caminho $0 \rightarrow 7 \rightarrow 9$ com peso 180. Ou seja, ele terminou cedo demais e perdeu quase todo o grafo.

Por que o guloso erra: a gente quer maximizar a soma de um caminho que não repete vértice. Mas escolher sempre a maior aresta no momento pode (1) levar cedo demais ao destino e encerrar o caminho curto, ou (2) gastar um vértice que faria falta numa rota bem maior mais pra frente. Como o guloso nunca volta atrás, ele fica preso nesse ótimo local. Esse problema simplesmente não tem a propriedade de escolha gulosa que faz heurísticas assim funcionarem em outros problemas — o que faz sentido, já que ele é NP-Difícil.

Resumindo o trade-off

- **Exato:** sempre acha o melhor caminho, mas o custo cresce como $(n-2)!$ e fica inviável já por volta de $n=12$ a 15 em grafos completos.
- **Guloso:** é rapidíssimo ($O(n^2)$ no pior caso) e sempre devolve alguma rota, mas sem garantia nenhuma de qualidade — pode errar muito e às vezes nem chega no destino.

Referências Bibliográficas

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009.
2. GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.
3. SIPSER, M. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2013.
4. KARP, R. M. Reducibility Among Combinatorial Problems. In: MILLER, R. E.; THATCHER, J. W. (Eds.). *Complexity of Computer Computations*. New York: Plenum Press, 1972. p. 85-103.
5. LIMA, G. *Notas e slides de aula — Programação Dinâmica, Algoritmos Gulosos, Backtracking e Branch and Bound, e Introdução às classes P, NP e NP-Completo*. Instituto de Computação, UFBA, 2025.
6. HUNTER, J. D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90-95, 2007.